BANCO DE DADOS · SISTEMAS WEB

Aula 21
Estrutura de Arquivos para Sistemas Web

Professor: Moisés Sousa

Objetivos da aula

Ao final, você será capaz de:

- Reconhecer uma estrutura de projeto desorganizada
- Organizar um projeto em public/, config/ e app/
- Explicar o papel de Model, View e Controller
- Relacionar tabela, Model, CRUD e SQL
- Aplicar 4 práticas de segurança na estrutura

O problema da desorganização

Sexta-feira, 18h. Bug em produção. Em qual dos 7 arquivos está a regra?

Manutenção

Arquivos perdidos, nomes sem sentido.

Bugs

Código duplicado e regras misturadas.

Segurança

Arquivos sensíveis expostos.

Equipe

Conflitos e retrabalho.

Estrutura bagunçada, anotada

```
sistema/
├── conexao.php
├── salvar.php
├── editar.php
├── excluir.php
├── teste2.php
├── loginnovo.php
└── backup.php
```

- **teste2.php, loginnovo.php:** nomes sem sentido, versões duplicadas
- **backup.php:** baixável por URL
- **conexao.php:** incluído em cada arquivo
- HTML, SQL e regras no mesmo arquivo

Separação de responsabilidades

Analogia do restaurante: cada um faz uma coisa.

Restaurante	Sistema web	Papel
Cliente	Usuário	Faz o pedido
Garçom	Controller	Recebe, coordena, entrega
Cozinha	Model	Prepara os dados
Estoque	Banco de dados	Guarda os dados
Prato	View	O que o cliente vê

Estrutura profissional

```
sistema/
├── public/          ← única pasta exposta
│   ├── index.php    ← ponto de entrada
│   └── css/ js/ img/
├── config/          ← configurações
├── app/
│   ├── controllers/  ← orquestra
│   ├── models/       ← dados e regras
│   └── views/        ← HTML
├── uploads/         ← arquivos enviados
├── logs/            ← erros
├── vendor/          ← bibliotecas (Composer)
├── .env             ← senhas, fora do Git
└── .gitignore
```

Pasta public/

O servidor web aponta para public/ (document root). Por isso config/ e app/ não têm URL.

- index.php: porta única de entrada
- CSS, JavaScript e imagens

Pergunta: o que acontece se alguém digitar /config/ na URL?

Pasta config/

Configurações lidas do .env, que nunca vai para o Git.

```
$db_host = $_ENV['DB_HOST'];
$db_name = $_ENV['DB_NAME'];
$db_user = $_ENV['DB_USER'];  // privilégios mínimos, nunca root
$db_pass = $_ENV['DB_PASS'];  // vem do .env
```

Corrigimos o exemplo anterior: usuário root e senha no código contradiz a boa prática.

Introdução ao MVC

Model

Representa os dados e conversa com o banco.

View

Mostra o resultado ao usuário. Sem SQL.

Controller

Recebe a requisição, chama o Model e escolhe a View.

Fluxo do MVC

Ida



Volta



Model: exemplo

```
class UsuarioModel {
  public function __construct(private PDO $pdo) {}

  public function listar(): array {
    return $this->pdo->query(
      "SELECT id, nome, email FROM usuarios"
    )->fetchAll(PDO::FETCH_ASSOC);
  }

  public function salvar(string $nome, string $email): void {
    $sql = "INSERT INTO usuarios (nome, email)
            VALUES (:nome, :email)";
    $this->pdo->prepare($sql)
         ->execute([':nome'=>$nome, ':email'=>$email]);
  }
}
```

Prepared statements previnem SQL injection.

Controller: exemplo

```
class UsuarioController {
  public function index() {
    $model = new UsuarioModel(Database::conectar());
    $usuarios = $model->listar();   // pede ao Model
    require __DIR__ . '/../views/usuarios.php'; // entrega à View
  }
}
```

O Controller orquestra e não contém SQL.

View: exemplo

```
<?php foreach ($usuarios as $u): ?>
  <li><?= htmlspecialchars($u['nome']) ?></li>
<?php endforeach; ?>
```

- View não tem SQL nem regra de negócio
- htmlspecialchars evita XSS

Tabela, Model, CRUD e SQL

Tabela	Arquivo	Função	SQL
usuarios	UsuarioModel.php	listar()	SELECT
usuarios	UsuarioModel.php	salvar()	INSERT
usuarios	UsuarioModel.php	editar()	UPDATE
usuarios	UsuarioModel.php	excluir()	DELETE

Chaves estrangeiras (ex.: matriculas → usuarios, cursos) viram consultas com JOIN no Model correspondente.

Segurança na estrutura

- Só public/ exposta; .env e config/ fora dela
- Usuário do banco com privilégios mínimos
- Uploads: validar tipo e tamanho, renomear, bloquear execução de PHP
- logs/ fora da web; erros detalhados desligados em produção
- Prepared statements e escape de saída

Erros comuns

- SQL dentro da View
- Regra de negócio duplicada no Model e no Controller
- .env enviado ao Git
- backup.php ou phpinfo.php público
- root na conexão com o banco

Atividade prática

Em dupla, 30 a 40 minutos: refatore a estrutura bagunçada.

- Crie a árvore de pastas
- Mova cada arquivo para o lugar certo
- Renomeie loginnovo.php
- Separe a conexão em config/

Entrega: print da árvore e uma justificativa de uma linha por arquivo movido.

Conclusão

Organização sustenta:

- **Manutenção**
- **Segurança**
- **Crescimento**
- **Equipe**

Próxima aula: CRUD completo seguindo essa estrutura.